Foundations of Spatial AI

Lecture 09 — Neural Implicit Rendering

Lecture notes

Based on the slides by E. Giacomini and L. Di Giammarino
Sapienza University of Rome, October 21, 2026

How to read these notes. Sections follow the slide deck; the slide range is shown in each header. Formal statements are in coloured boxes, blue *Intuition* boxes explain them in plain words, and dashed *placeholder* boxes indicate which slide image to paste there (photos, renderings and videos are not redrawn). All diagrams drawn here are original TikZ/pgfplots figures. Only the slide deck was provided (no companion PDF), so results the slides only state informally are made precise inside the main flow; the Appendix contains supplementary material that is *not* on the slides.

Contents

1 Neural Implicit Rendering *(slides 1–2)* **3**

2 Review: Volume Rendering *(slides 3–4)* **3**

3 (Neural) Volumes: A Very Expressive Representation *(slide 5)* **4**

4 Back to Surfaces *(slides 6–8)* **5**

5 Recap: 3D Representations *(slide 9)* **6**

6 Recap: Implicit Surfaces *(slide 10)* **6**

7 Neural Implicit Surfaces *(slides 11–12)* **6**

8 Rendering Neural Implicit Surfaces *(slides 13–14)* **7**

9 Intersections of Rays with Implicit Surfaces *(slides 15–22)* **8**

9.1 Problem statement *(slides 15–16)* 8

9.2 Searching for roots of a 1-d function: secant method *(slides 17–18)* 8

9.3 Intersections of rays with SDFs: sphere tracing *(slides 19–20)* 9

9.4 Options and differentiability *(slides 21–22)* 10

10 Differentiable Root Finding *(slides 23–26)* **10**

10.1 A toy example *(slide 23)* 10

10.2 The general rule *(slides 24–26)* 11

11 Differentiable Ray Marching ***(slides 27–29)*** **11**

12 Rendering Implicit Surfaces (or SDFs) ***(slides 30–32)*** **12**

13 Learning Implicit Geometry and Appearance ***(slide 33)*** **12**

14 Rendering Neural Implicits ***(slides 34–36)*** **13**

15 Rendering Neural Implicits: Practical Considerations ***(slides 37–38)*** **13**

16 Visibility Discontinuities ***(slide 39)*** **14**

17 Rendering Implicit Surfaces ***(slide 40)*** **15**

18 Volume Rendering for Implicit Surfaces ***(slides 41–43, 49)*** **15**

19 From SDF to Density ***(slides 44–48)*** **16**

19.1 A first choice: VolSDF *(slide 44)* 16

19.2 A second heuristic: NeuS *(slides 45–46)* 16

19.3 Comparison of the densities *(slides 47–48)* 17

20 Volume Rendering for Implicit Surfaces: results ***(slides 50–54)*** **18**

21 Summary ***(slides 55–56)*** **20**

A Appendix: supplementary material (not on the slides) **21**

A.1 From the continuous volume rendering integral to Eq. (1) 21

A.2 Ψ_β is a Laplace CDF 21

1 Neural Implicit Rendering *(slides 1–2)*

The lecture asks how to *render* (and learn from images) a 3D scene whose geometry is stored as a neural *implicit surface* — the zero level set of a neural network — rather than as a density volume (NeRF). The running example on the slides is a signed distance field (SDF) of a teapot: white iso-contours around the object, with a ray (red arrow) shot towards the surface.

[PLACEHOLDER IMAGE]
Insert the image from **slide 2 (also slides 11–13, 15–16, 21–22, 27, 29–30, 32, 37–38, 40–41)**
Teapot with SDF iso-contours and a query ray (Takikawa et al., NGLOD)

Intuition

Three questions organise the whole lecture (they are stated explicitly on the "Today's Goals" slide): (1) *where does a ray hit the surface, and can we differentiate that?* (2) *what colour do we assign to a surface point?* (3) *can we get the best of surfaces and volumes by turning an implicit surface into a density?*

2 Review: Volume Rendering *(slides 3–4)*

A pixel is not a single radiance sample

A pixel value is proportional to the radiance $L(\boldsymbol{x}^*, \boldsymbol{\omega})$ arriving at the sensor centre $\boldsymbol{x}^*$ from direction $\boldsymbol{\omega}$ (pointing from the scene point towards the optical centre). The slide stresses that this is *not* the radiance at one point: $L(\boldsymbol{x}^*, \boldsymbol{\omega}) \neq L(\boldsymbol{x}^* - \lambda\boldsymbol{\omega}, \boldsymbol{\omega})$, because along the ray the light is attenuated and emitted by the participating medium (the cloud in the figure). *The pixel value cannot be reduced to a radiance at a single point.*

Discrete volume rendering

Sample the ray at $\boldsymbol{x}_{t_i}$, $i = 1, \dots, N$, with spacing Δt, density $\sigma_{t_i} = \sigma(\boldsymbol{x}_{t_i})$ and emitted radiance $L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})$:

$$L(\boldsymbol{x}, \boldsymbol{\omega}) = \sum_{i=1}^{N} T(\boldsymbol{x}, \boldsymbol{x}_{t_i}) \left(1 - e^{-\sigma_{t_i} \Delta t}\right) L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega}), \qquad T(\boldsymbol{x}, \boldsymbol{x}_{t_i}) = T(\boldsymbol{x}, \boldsymbol{x}_{t_{i-1}})\, e^{-\sigma_{t_{i-1}} \Delta t}. \tag{1}$$

The slide's message: we must **combine the contributions of multiple samples/segments along a ray**.

Lemma 2.1: Rendering weights telescope

Let $T_i := T(\boldsymbol{x}, \boldsymbol{x}_{t_i})$ and $w_i := T_i \left(1 - e^{-\sigma_{t_i} \Delta t}\right)$ be the weight of sample i in (1). Then $T_i = \exp\left(-\sum_{j<i} \sigma_{t_j} \Delta t\right)$, $w_i = T_i - T_{i+1}$ and hence $\sum_{i=1}^{N} w_i = 1 - T_{N+1} \leq 1$.

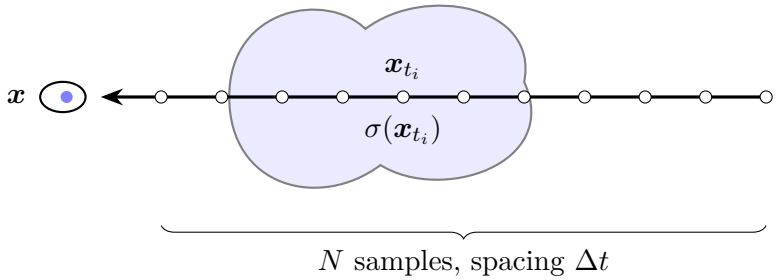


Figure 1: Redrawn from slide 4: samples $\boldsymbol{x}_{t_i}$ along the ray through a medium with density σ.

Proof. Unrolling the recursion with $T_1 = 1$ gives the exponential of the cumulative optical depth. Then $w_i = T_i - T_i e^{-\sigma_{t_i}\Delta t} = T_i - T_{i+1}$, and the sum telescopes. □

Intuition

T_i is the fraction of light that survives until sample i; w_i is the fraction that is *absorbed (or emitted) exactly in segment* i. Example with $\Delta t = 1$ and $\sigma = (0, 0.5, 2, 2)$: $T = (1, 1, 0.607, 0.082, 0.011)$ and $w = (0, 0.394, 0.524, 0.071)$, whose sum is $0.989 = 1 - T_5$. Almost all weight lies on the first dense segments: the ray "stops" where the medium becomes opaque.

3 (Neural) Volumes: A Very Expressive Representation *(slide 5)*

Paraphrasing the slide: *with great representation capacity comes greater risk of overfitting and local optima.* A neural density/radiance field can explain the training images in many ways, most of which are not physically sensible shapes. The slide shows NeRS (Zhang et al., NeurIPS 2021) reconstructions of two cars from few views, where volumes collapse to blurry, view-inconsistent blobs.

[PLACEHOLDER IMAGE]
Insert the image from **slide 5**
Photos of a red pick-up and a blue SUV with
their poor volumetric reconstructions (NeRS)

Intuition

A volume has one free number per point in space (density) plus colour; nothing forces the density to concentrate on a 2D surface. Fitting a handful of images is under-constrained, so the optimiser may pick a "cloud" that reproduces the pixels but is not the object.

4 Back to Surfaces *(slides 6–8)*

A sample result (slide 6)

NeRF-like representations do not necessarily capture geometry well: the extracted NeRF surface is noisy, while the NeuS geometry (Wang et al., NeurIPS 2021) is smooth, with a faithful rendering.

[PLACEHOLDER IMAGE]
Insert the image from **slide 6**
NeRF geometry vs. "our geometry" and "our rendering" (NeuS)

Training in a few seconds (slide 7)

The slide shows a video of Instant-NGP (Müller et al., SIGGRAPH 2022): eight scenes being trained, with an "elapsed training time" counter (0 seconds at the start).

[PLACEHOLDER IMAGE]
Insert the image from **slide 7**
Video frame, Instant-NGP training grid (elapsed time 0 s)

Key idea (slide 8)

The classical NeRF scenes (chair, lego, materials, drums, mic, hotdog, ship, ficus) are *all examples of scenes with well-defined surfaces (except the water).*

Key idea

Let's **enforce that the geometry corresponds to shapes with (well-defined) surfaces!**

[PLACEHOLDER IMAGE]
Insert the image from **slide 8**
Faded NeRF synthetic scenes behind the key-idea text (Mildenhall et al.)

Intuition

If the real world is made of surfaces, a representation that can *only* express surfaces has a smaller hypothesis space, so fewer wrong explanations of the images — at the price of harder optimisation, as we will see.

5 Recap: 3D Representations *(slide 9)*

Representation	Weakness quoted on the slide
Point cloud	Difficult to extract a surface
Mesh (triangles, parametric patches)	Difficult to capture geometry with arbitrary topology and fine details (e.g. leaves, slats in chairs)
Voxel grid	Surface via marching cubes, but resolution is limited
Implicit surface	(the representation advocated next)

[PLACEHOLDER IMAGE]
Insert the image from **slide 9**
Four representations: point cloud (cow), mesh (bunny, saddle patch), voxels, implicit surface

6 Recap: Implicit Surfaces *(slide 10)*

Definition 6.1: Implicit surface

Given a continuous $f : \mathbb{R}^3 \to \mathbb{R}$, the *implicit surface* it parametrises is its zero level set $\mathcal{S} = \{\boldsymbol{p} \mid f(\boldsymbol{p}) = 0\}$.

f "can be anything — a simple function, or a neural network" (slide code: input $\boldsymbol{p} \in \mathbb{R}^3$, output a scalar).

Intuition

Think of f as an altitude map in 3D: the surface is the "sea level" $f = 0$, the region $f < 0$ is inside and $f > 0$ is outside. Topology (holes, handles, several components) comes for free, and resolution is limited only by the function, not by a grid.

7 Neural Implicit Surfaces *(slides 11–12)*

Take f_θ to be a neural network with parameters θ: $\{\boldsymbol{p} \mid f_\theta(\boldsymbol{p}) = 0\}$ *allows representing complex geometry.* Slide question: *what additional constraint is required for a signed distance field?* Answer (slide 12): $\|\nabla f\| = 1$.

Definition 7.1: Signed distance function (SDF)

For a closed surface $\mathcal{S}$ bounding a region Ω, the SDF is $d_{\mathcal{S}}(\boldsymbol{p}) = \pm \min_{\boldsymbol{y} \in \mathcal{S}} \|\boldsymbol{p} - \boldsymbol{y}\|$, with the sign convention used in the sphere-tracing slides: positive outside, negative inside.

Proposition 7.2: Eikonal property

Wherever an SDF is differentiable, $\|\nabla d_{\mathcal{S}}\| = 1$.

Proof. $d_{\mathcal{S}}$ is 1-Lipschitz (moving by h changes the distance to the closest surface point by at most h), so $\|\nabla d\| \leq 1$. Conversely, moving from $\boldsymbol{p}$ straight towards its closest surface point $\boldsymbol{y}$ by a step h decreases the distance by exactly h, so the directional derivative in that direction has magnitude 1 and $\|\nabla d\| \geq 1$. □

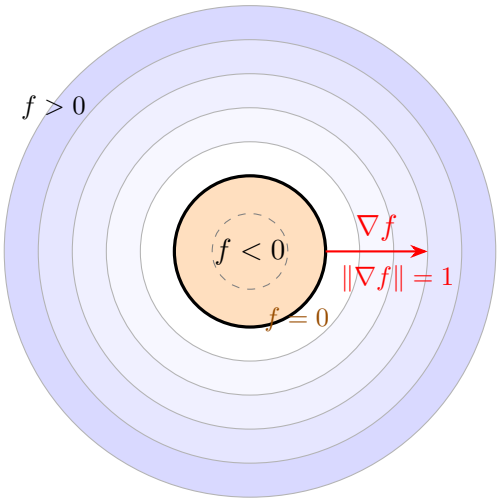


Figure 2: Original illustration: SDF of a disc. Iso-contours are equally spaced because $\|\nabla f\| = 1$.

Intuition

The iso-contours of an SDF are evenly spaced like the rings of a ripple: the value of f *is* the distance to the surface, so it grows with unit rate. A generic neural f_θ has the right zero set but arbitrary spacing; the *eikonal* penalty $\big|\|\nabla f\| - 1\big|$ (slide 33) nudges the network towards this nicer behaviour. The slide links an optional read on why distance functions satisfy an eikonal equation (Daniel Duffy's blog post).

8 Rendering Neural Implicit Surfaces *(slides 13–14)*

The appearance at each pixel is determined by a *unique surface point*. This raises two questions: **where does a ray intersect an implicit surface?** and **how to model appearance?**

Today's goals (slide 14)

1. Computational methods for (differentiable) implicit surface–ray intersection.
2. Modelling appearance (today: simple neural models; later: physically informed models).
3. Unifying surfaces and volumes: volume rendering for implicit surface representations.

9 Intersections of Rays with Implicit Surfaces *(slides 15–22)*

9.1 Problem statement *(slides 15–16)*

Input: implicit representation $f_\theta(\cdot)$ and a query ray $\boldsymbol{r} : \boldsymbol{x}_0 + t\boldsymbol{\omega}$. **Output:** the intersection $\boldsymbol{p} \equiv t^*\boldsymbol{\omega} + \boldsymbol{x}_0$ such that $f_\theta(\boldsymbol{p}) = 0$ (for now assume the ray does hit the surface; revisited later). Defining the restriction $\bar{f}_\theta(t) \equiv f_\theta(\boldsymbol{x}_0 + t\boldsymbol{\omega})$, this is **a 1-d search problem!**

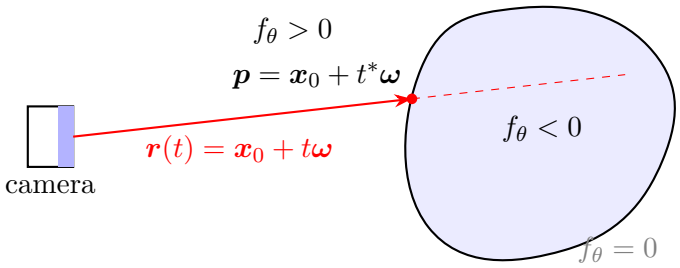


Figure 3: Original sketch for slides 13–16: the ray meets the zero level set of f_θ at t^*.

9.2 Searching for roots of a 1-d function: secant method *(slides 17–18)*

Algorithm 9.1: Secant method

Given (x_0, f_0) and (x_1, f_1) with $f_i = f(x_i)$, form the linear approximation

$$y_2 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x_2 - x_1) + f(x_1).$$

Its root $(y_2 = 0)$ is

$$x_2 = x_1 - f(x_1)\frac{x_1 - x_0}{f(x_1) - f(x_0)},$$

then repeat with (x_1, x_2).

It is a general method, applicable for any f; but it is *not guaranteed to converge if far away from the root (or if f is not "nice")*. Other methods (e.g. Newton's) could be used. **Can we do better if rendering an implicit SDF and not an arbitrary function f?**

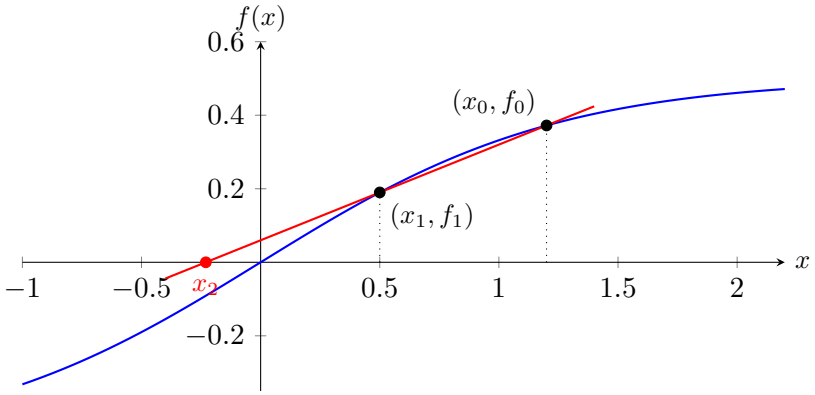


Figure 4: Original figure for slide 17: one secant step on $f(x) = 0.5\tanh(0.8x)$ with $x_0 = 1.2$, $x_1 = 0.5$ gives $x_2 \approx -0.23$ (true root 0).

Intuition

Replace the curve by the straight line through the last two points and jump to where that line crosses zero. It is fast when f is locally almost linear, but a wiggly neural f far from the root can send the iterate anywhere.

9.3 Intersections of rays with SDFs: sphere tracing *(slides 19–20)*

Slide 19 is a video (Sebastian Lague, "Coding Adventure: Ray Marching") showing a 2D scene of circles and boxes and circles growing around the current ray point.

[PLACEHOLDER IMAGE]
Insert the image from **slide 19**
Ray-marching video frame: dark background, circles/boxes, one white ray origin

Algorithm 9.2: Sphere tracing

Initialise t (e.g. 0) and $\boldsymbol{p} = \boldsymbol{x}_0 + t\boldsymbol{\omega}$. While $f(\boldsymbol{p}) > \epsilon$:

$$t \leftarrow t + f(\boldsymbol{p}), \qquad \boldsymbol{p} \leftarrow \boldsymbol{x}_0 + t\boldsymbol{\omega},$$

stopping also if a maximum number of steps is exceeded. Basically: *walk along the ray until close to a surface.*

Lemma 9.3: Sphere tracing never crosses the surface

If f is an SDF, the open ball $B(\boldsymbol{p}, f(\boldsymbol{p}))$ contains no surface point.

Proof. If $\boldsymbol{y} \in \mathcal{S}$ with $\|\boldsymbol{y} - \boldsymbol{p}\| < f(\boldsymbol{p}) = \min_{\boldsymbol{z}\in\mathcal{S}} \|\boldsymbol{p} - \boldsymbol{z}\|$, the minimum would be violated. □

Proposition 9.4: Convergence to the first intersection

Let f be an SDF, the start point be outside, and t^* the first hit of the ray. The sphere-tracing iterates t_k are non-decreasing, bounded by t^*, and converge to t^* (as $\epsilon \to 0$, with unlimited steps).

Proof. Because $\boldsymbol{p}^*$ is a surface point at distance $t^* - t_k$ from $\boldsymbol{p}_k$, $f(\boldsymbol{p}_k) \le t^* - t_k$ and so $t_{k+1} = t_k + f(\boldsymbol{p}_k) \le t^*$. The sequence is monotone and bounded, hence convergent to some $t_\infty \le t^*$; increments $f(\boldsymbol{p}_k) = t_{k+1} - t_k \to 0$, so by continuity $f(\boldsymbol{p}_\infty) = 0$, i.e. $\boldsymbol{p}_\infty \in \mathcal{S}$. Since $t_\infty \le t^*$ and t^* is the first hit, $t_\infty = t^*$. □

Intuition

The SDF tells you "nothing is closer than $f(\boldsymbol{p})$", so you can safely leap that far along *any* direction. Far from the surface the leaps are long, close to it they shrink geometrically, so

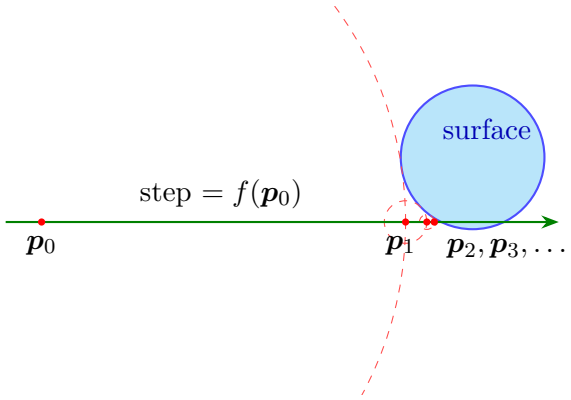


Figure 5: Original illustration of sphere tracing (slide 20) on a disc of radius 1 centred at $(6, 0.9)$: radii of the safe balls are 5.07, 0.30, 0.10, 0.05, ... and the iterates approach the first hit without crossing it.

you creep up on the surface from outside and never tunnel through. Slide question *Q: why will we never cross the surface?* — Lemma 9.3.

9.4 Options and differentiability *(slides 21–22)*

Two ways to get $\boldsymbol{p} \equiv t^*\boldsymbol{\omega} + \boldsymbol{x}_0$ s.t. $f_\theta(\boldsymbol{p}) = 0$: **Option 1:** a generic root-finding method (e.g. secant); **Option 2:** sphere tracing (only if rendering SDFs). Next question: *is this differentiable? Do we need to backpropagate through the iterative process?*

10 Differentiable Root Finding *(slides 23–26)*

10.1 A toy example *(slide 23)*

Let $f_\theta(t) = t^2 - \theta$ (plotted for $\theta = 4$); some root-finder returns $t^* = 2$. If θ changes, the new root is $t^*(\theta + \Delta\theta) \equiv t^* + \Delta t$. At the old root $f_{\theta+\Delta\theta}(t^*) = -\Delta\theta$, so *the root must change to compensate for the change in* f:

$$\Delta t \approx \frac{\Delta\theta}{2t^*} \qquad (\textit{why?})$$

Because $f_{\theta+\Delta\theta}$ near t^* is approximately $-\Delta\theta + 2t^*(t - t^*)$, which vanishes at $\Delta t = \Delta\theta/(2t^*)$.

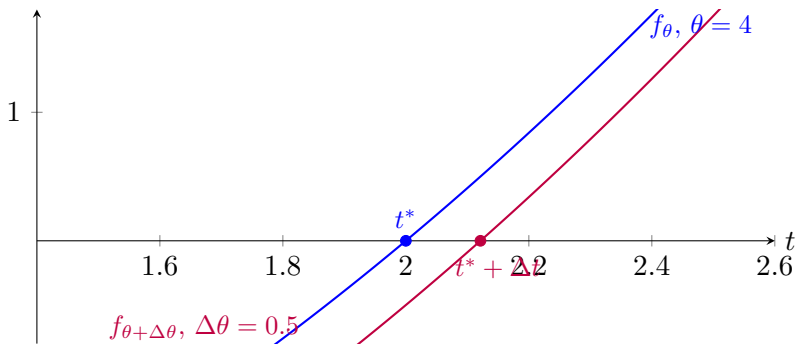


Figure 6: Original figure for slide 23: perturbing θ from 4 to 4.5 shifts the root from 2 to $\sqrt{4.5} \approx 2.121$; the linear prediction $\Delta\theta/(2t^*) = 0.125$ vs. the true shift 0.121.

10.2 The general rule *(slides 24–26)*

For a general f_θ, $f_{\theta+\Delta\theta}(t^*) \approx \frac{\partial f_\theta(t^*)}{\partial \theta}\Delta\theta$ and $\Delta t \approx -f_{\theta+\Delta\theta}(t^*)\left(\frac{\partial f_\theta(t^*)}{\partial t}\right)^{-1}$, i.e.,

$$\Delta t \approx -\Big(\frac{\partial f_\theta(t^*)}{\partial t}\Big)^{-1}\frac{\partial f_\theta(t^*)}{\partial \theta}\Delta\theta.$$

Theorem 10.1: Implicit differentiation of a root

Let $F(\theta, t) = f_\theta(t)$ be differentiable and $t^*(\theta)$ a root, $F(\theta, t^*(\theta)) = 0$, with $\frac{\partial F}{\partial t}(\theta, t^*) \neq 0$. Then

$$\frac{\partial t^*}{\partial \theta} = -\Big(\frac{\partial f_\theta(t^*)}{\partial t}\Big)^{-1}\frac{\partial f_\theta(t^*)}{\partial \theta}.$$

This holds irrespective of how t^* was found, and for any f.

Proof. By the implicit function theorem $t^*(\theta)$ is locally differentiable. Differentiate $F(\theta, t^*(\theta)) = 0$ with the chain rule: $\frac{\partial F}{\partial \theta} + \frac{\partial F}{\partial t}\frac{\partial t^*}{\partial \theta} = 0$ and solve. □

Intuition

If changing the parameters changes the value of f at the current root, the root must move to put the value back to zero. How far it moves is the "error" $\partial f/\partial\theta$ divided by how steeply f crosses zero, $\partial f/\partial t$. *Numbers:* for the toy example, $\partial f/\partial\theta = -1$ and $\partial f/\partial t = 2t^* = 4$, so $\partial t^*/\partial\theta = 1/4$, matching $\Delta t \approx \Delta\theta/(2t^*)$. Note a steep crossing (large $|\partial f/\partial t|$) makes the root insensitive to θ, while a grazing crossing ($\partial f/\partial t \to 0$) makes it explode — the first sign of the problems with silhouettes below.

11 Differentiable Ray Marching *(slides 27–29)*

Apply the theorem to $\bar{f}_\theta(t) \equiv f_\theta(\boldsymbol{x}_0 + t\boldsymbol{\omega})$ with intersection $\boldsymbol{p} \equiv t^*\boldsymbol{\omega} + \boldsymbol{x}_0$. (Camera origin $\boldsymbol{x}_0$ and direction $\boldsymbol{\omega}$ do not depend on θ.)

Corollary 11.1: Analytic gradient of the intersection point

Differentiating $f_\theta(\boldsymbol{p}) = 0$ gives

$$\frac{\partial f_\theta(\boldsymbol{p})}{\partial \theta} + \frac{\partial f_\theta(\boldsymbol{p})}{\partial \boldsymbol{p}} \cdot \frac{\partial \boldsymbol{p}}{\partial \theta} = 0, \qquad \frac{\partial \boldsymbol{p}}{\partial \theta} = \boldsymbol{\omega}\,\frac{\partial t^*}{\partial \theta},$$

hence

$$\frac{\partial t^*}{\partial \theta} = -\Big(\frac{\partial f_\theta(\boldsymbol{p})}{\partial \boldsymbol{p}} \cdot \boldsymbol{\omega}\Big)^{-1}\frac{\partial f_\theta(\boldsymbol{p})}{\partial \theta}, \qquad \frac{\partial \boldsymbol{p}}{\partial \theta} = -\boldsymbol{\omega}\Big(\frac{\partial f_\theta(\boldsymbol{p})}{\partial \boldsymbol{p}} \cdot \boldsymbol{\omega}\Big)^{-1}\frac{\partial f_\theta(\boldsymbol{p})}{\partial \theta}.$$

If we can find an intersection, we can analytically compute $\partial\boldsymbol{p}/\partial\theta$ — with no need to store (or backpropagate through) the iterations of the root finder.

Intuition

The forward pass can be as sloppy and iterative as you like (secant, sphere tracing, hundreds of steps); the backward pass only needs $\nabla_{\boldsymbol{p}} f$, $\partial f/\partial\theta$ and $\boldsymbol{\omega}$ *at the final point.* Memory does not grow with the number of marching steps. The dot product $\nabla_{\boldsymbol{p}} f \cdot \boldsymbol{\omega}$ is the cosine between

ray and surface normal (times $\|\nabla f\|$): head-on hits are stable, grazing hits are not.

12 Rendering Implicit Surfaces (or SDFs) *(slides 30–32)*

Step 1: Compute the intersection (root finding or sphere tracing for SDFs; analytically differentiable). **Step 2: Render the surface point:** $g_\phi(\boldsymbol{p}, \boldsymbol{\omega}) = \boldsymbol{c} \in \mathbb{R}^3$ (similar to NeRF), but slide 31 argues that *rendering should account for the surface normal in addition to the viewpoint*: two surface points with the same position and view direction but different normals reflect light differently. Hence (slide 32)

$$\boldsymbol{n} = \frac{\partial f_\theta(\boldsymbol{p})}{\partial \boldsymbol{p}}, \qquad g_\phi(\boldsymbol{p}, \boldsymbol{\omega}, \boldsymbol{n}) = \boldsymbol{c} \in \mathbb{R}^3.$$

(In practice $\boldsymbol{n}$ is normalised; for an SDF, $\|\nabla f\| = 1$ already.)

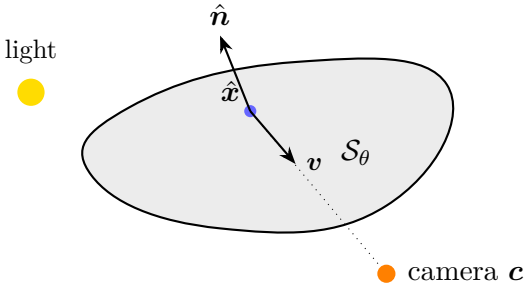


Figure 7: Original sketch of slide 31: the colour at $\hat{\boldsymbol{x}}$ depends on the normal $\hat{\boldsymbol{n}}$ and on the viewing direction $\boldsymbol{v}$ (the slide varies each in turn).

[PLACEHOLDER IMAGE]
Insert the image from **slide 31**
Three copies of the shape $\mathcal{S}_\theta$ with changed normal / viewing direction (Yariv et al., IDR)

Intuition

The geometry of a shiny object changes colour with orientation, and the only thing that tells the colour network about orientation is $\boldsymbol{n}$. Feeding $\boldsymbol{n} = \nabla f$ also *couples* appearance and geometry: errors in colour send gradients into f_θ through $\boldsymbol{n}$ and through $\boldsymbol{p}$.

13 Learning Implicit Geometry and Appearance *(slide 33)*

Given images I_i with camera poses π_i, fit geometry f_θ and appearance g_ϕ:

$$\min_{\theta,\phi} \big\|\text{render}(f_\theta, g_\phi, \pi_i) - I_i\big\| \; + \; \lambda \,\big|\, \|\nabla f\| - 1 \,\big|, \tag{2}$$

a photometric term plus the eikonal regulariser (the slide writes it without a weight; λ is the usual trade-off coefficient).

Intuition

The first term says "rendered pixels must match photographs" and drives *both* networks through the analytic gradients of the previous section; the second keeps f_θ a well-behaved distance field, which is what makes sphere tracing (and the normals) trustworthy.

14 Rendering Neural Implicits *(slides 34–36)*

Slides 34–36 are results of IDR (Yariv et al., NeurIPS 2020): geometry/appearance pairs for statues, fruits, cans, a Smurf toy, ...; a fountain reconstructed *using only 11 images*; and a training-run video with camera frusta around a skull while the SDF cross-section evolves.

[PLACEHOLDER IMAGE]
Insert the image from **slide 34**
Grid of reconstructed shapes (grey) and their textured renderings (IDR)

[PLACEHOLDER IMAGE]
Insert the image from **slide 35**
Fountain: geometry and rendering from only 11 images (IDR)

[PLACEHOLDER IMAGE]
Insert the image from **slide 36**
IDR training run: camera frusta around a skull, SDF slice on a blue plane

15 Rendering Neural Implicits: Practical Considerations *(slides 37–38)*

Q1: sphere tracing with neural implicits?

There is *no guarantee* that a trained f_θ is an actual SDF. Lemma 9.3 then fails: if $|f_\theta|$ overestimates the distance the march can step through a surface; if it underestimates it is merely slow. **In practice, combine with the secant method to handle "bad" rays.** And crucially: analytic differentiation (Corollary 11.1) *still holds no matter how we compute the intersection.*

Algorithm 15.1: A typical hybrid intersection routine

(Typical implementation, not spelled out on the slides.)

1. Sphere trace with step $f_\theta(\boldsymbol{p})$ (optionally clamped) up to a maximum number of steps.
2. If f_θ changes sign between two iterates t_{k-1}, t_k (the ray stepped through the surface), keep that bracket.
3. Refine the root inside the bracket with secant (or bisection) iterations.
4. Rays with no sign change are declared *non-intersecting*.

Q2: rays that do not intersect

There is no rendering loss on those pixels, but it is **crucial to use a mask loss**. If a pixel is foreground but the ray did not intersect:

$$L(\boldsymbol{r}) \equiv \min_t f_\theta(\boldsymbol{x}_0 + t\boldsymbol{\omega}) \qquad \text{(SDF of the closest surface point should be 0)},$$

implemented in practice by uniformly sampling t.

Intuition

Without a mask, the optimiser can "cheat": move the surface so that rays miss it and hence incur no photometric loss. The mask loss says: the ray that *should* touch the object must at least come as close as possible to the surface; the gradient flows through the best sample t and pulls the surface towards the ray. (By symmetry, a background pixel whose ray *does* hit the surface must be penalised too — standard in IDR.)

16 Visibility Discontinuities *(slide 39)*

Correctly differentiating the surface rendering equation requires accounting for *visibility discontinuities*; this is the main focus of differentiable-rendering research in graphics. The slide shows the silhouette/horizon/terminator geometry of a surface seen from a camera (Gargallo et al., ICCV 2007) and a light-transport example where a *silhouette edge* lies on a ray path.

[PLACEHOLDER IMAGE]
Insert the image from **slide 39**
Left: banana with horizon, terminator and crepuscular cone; right: sphere and Cornell-box silhouette edge

Intuition

The pixel colour is a piecewise-smooth function of the shape: smooth while the ray keeps hitting the same part of the surface, but with a *jump* when the ray crosses an object boundary. Differentiating "inside" each piece is easy (everything above); the gradient due to the boundary moving across the pixel is the hard part, and is why accurate differentiable rendering is non-trivial.

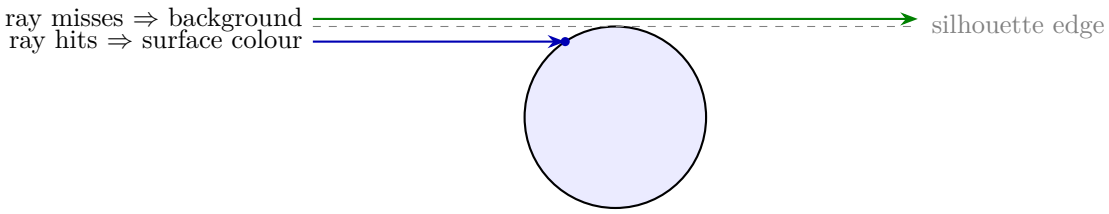


Figure 8: Original toy figure: an arbitrarily small change of θ moves a ray from just inside to just outside the silhouette, so the pixel value jumps. Implicit differentiation (Corollary 11.1) captures only the smooth part of the derivative, not this jump.

17 Rendering Implicit Surfaces *(slide 40)*

Implicit surfaces: a nice representation for geometry. **Implicit surface rendering:** render a *single* point per ray.

- \+ closer to physical reality
- − makes optimisation difficult (e.g. mask loss is critical)
- \+ can allow physics-based rendering using surface normals (later lecture)
- − accurate differentiable rendering is non-trivial due to visibility discontinuities

18 Volume Rendering for Implicit Surfaces *(slides 41–43, 49)*

Idea (slide 41): convert the implicit into a "density" and use volume rendering!

Recall (slide 42): a rendering algorithm maps $(\sigma_{t_i} \equiv \sigma(\boldsymbol{x}_{t_i}),\ L_e(\boldsymbol{x}_{t_i}, \boldsymbol{\omega})) \to L(\boldsymbol{x}, \boldsymbol{\omega})$ and a neural density/radiance representation $(\boldsymbol{x}, \boldsymbol{\omega}) \to f_\theta \to (\sigma, \boldsymbol{c})$, $\sigma \in \mathbb{R}^+$, $\boldsymbol{c} \in [0,1]^3$. For implicit surfaces (slides 43, 49) the network outputs a signed distance $d \in \mathbb{R}$ and we *analytically convert* it to density with a map Ψ, using volume rendering while enforcing the SDF prior $\big|\|\nabla f\| - 1\big|$.

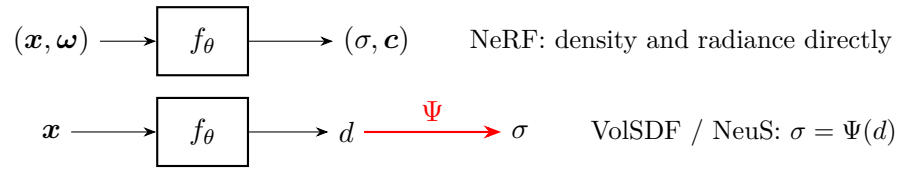


Figure 9: Redrawn from slides 42–43/49: the density is produced by a fixed analytic map Ψ of the signed distance.

Intuition

The renderer (Eq. (1)) is untouched. We only change *what the network predicts*: instead of free density we predict a distance and let a formula decide the density. Geometry is then a surface by construction, and optimisation keeps the soft, forgiving gradients of volume rendering (no mask loss, no single-point visibility problem).

19 From SDF to Density *(slides 44–48)*

19.1 A first choice: VolSDF *(slide 44)*

Definition 19.1: VolSDF density

With d_Ω the signed distance (negative inside), $\alpha, \beta > 0$:

$$\sigma(\boldsymbol{x}) = \alpha\, \Psi_\beta(-d_\Omega(\boldsymbol{x})), \qquad \Psi_\beta(s) = \begin{cases} \frac{1}{2}\exp(\frac{s}{\beta}) & s \le 0, \\ 1 - \frac{1}{2}\exp(-\frac{s}{\beta}) & s > 0. \end{cases}$$

Outside $= 0$, inside $= \alpha$, with sharpness of the drop controlled by $1/\beta$. *Just a heuristic — one among many!*

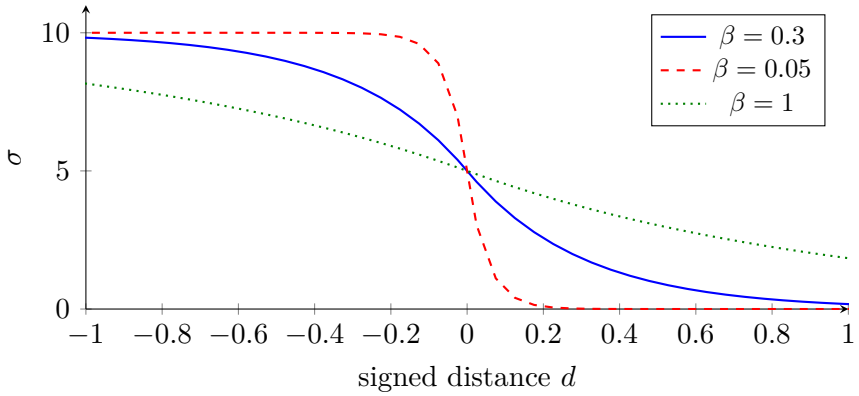


Figure 10: Original plot of the VolSDF density for $\alpha = 10$: small β approaches a sharp step at the surface, large β gives a fuzzy shell.

Proposition 19.2: Ψ_β is a Laplace CDF

Ψ_β is the cumulative distribution function of a zero-mean Laplace variable Z with scale β, so $\sigma(\boldsymbol{x}) = \alpha \Pr[Z \ge d_\Omega(\boldsymbol{x})]$. (Proof: see Appendix A.2; supplementary.)

Intuition

Imagine the surface is randomly "jittered" outward or inward by a Laplace-distributed amount Z; the density at $\boldsymbol{x}$ is α times the probability that $\boldsymbol{x}$ ends up inside. Points deep inside are always inside ($\sigma = \alpha$), points far outside never ($\sigma = 0$), and at the surface it is a coin flip ($\sigma = \alpha/2$).

19.2 A second heuristic: NeuS *(slides 45–46)*

Take the logistic density and sigmoid

$$\phi_s(x) = \frac{s\, e^{-sx}}{(1 + e^{-sx})^2}, \qquad \Phi_s(x) = (1 + e^{-sx})^{-1}, \qquad \Phi_s' = \phi_s.$$

The *naive* solution (slide 45) sets $\sigma(t) = \phi_s(f(\boldsymbol{p}(t)))$: high density around the surface and 0 otherwise, with sharpness controlled by s. *Yet another heuristic*; "the more principled one is

more complicated and involves gradients through f" — slide 46 shows the problem: *high weight before the surface.*

Proposition 19.3: Naive conversion is biased

Let $f(\boldsymbol{p}(t))$ be locally linear with a single zero at t^*, $\sigma = \phi_s(f(\boldsymbol{p}(t)))$, $T(t) = \exp(-\int_0^t \sigma)$ and $w = T\sigma$. Then $w'(t^*) < 0$: the weight peaks strictly *before* the surface.

Proof. $T' = -\sigma T$, so $w' = T'\sigma + T\sigma' = T(\sigma' - \sigma^2)$. At t^* the density $\phi_s(f)$ is maximal ($f = 0$), so $\sigma'(t^*) = 0$ and $w'(t^*) = -T\sigma^2 < 0$. □

Definition 19.4: NeuS opaque density

Let Φ_s be as above. NeuS defines

$$\rho(t) = \max\Big(\frac{-\frac{\mathrm{d}\Phi_s}{\mathrm{d}t}(f(\boldsymbol{p}(t)))}{\Phi_s(f(\boldsymbol{p}(t)))}, 0\Big)$$

and uses ρ in place of σ in volume rendering.

Proposition 19.5: Unbiased weights

Assume the ray crosses a locally planar surface from outside ($f > 0$) to inside, with $\nabla f \cdot \boldsymbol{\omega} = -\kappa < 0$ constant, and start far enough outside that $\Phi_s(f(0)) \approx 1$. Then $T(t) = \Phi_s(f(\boldsymbol{p}(t)))$ and the rendering weight is

$$w(t) = T(t)\rho(t) = \kappa\, \phi_s(f(\boldsymbol{p}(t))),$$

which is maximal exactly at the surface $f = 0$ ($t = t^*$).

Proof. Here $\frac{\mathrm{d}}{\mathrm{d}t}\Phi_s(f) = \phi_s(f)(\nabla f \cdot \boldsymbol{\omega}) = -\kappa\phi_s(f) < 0$, so $\rho = -\Phi_s(f)'/\Phi_s(f) > 0$ and $\int_0^t \rho = -\ln \Phi_s(f(\boldsymbol{p}(t))) + \ln \Phi_s(f(\boldsymbol{p}(0)))$. Hence $T = \Phi_s(f(t))/\Phi_s(f(0)) \approx \Phi_s(f(t))$ and $w = T\rho = -\frac{\mathrm{d}}{\mathrm{d}t}\Phi_s(f) = \kappa\phi_s(f)$, which peaks where ϕ_s does, at $f = 0$. □

Intuition

Naive: a bump of density *centred* on the surface, but light is already being absorbed on the way up the bump, so most of the pixel colour is taken from a slightly-in-front point: a systematic bias. NeuS instead designs ρ so that the *product* $T\rho$ is the bump we want, centred exactly at the zero crossing. Note the shape of ρ: it keeps increasing and then saturates inside ($\rho \to s\kappa$), i.e. the interior is opaque, whereas the naive σ returns to zero after the surface. *Number:* for $s = 10$ and a ray crossing at unit speed, the NeuS peak is $\phi_s(0) = s/4 = 2.5$ at t^*.

19.3 Comparison of the densities *(slides 47–48)*

Slide 47 shows a 2D SDF of a disc and a rectangle with a ray through both, and the density produced by NeRF (noisy blobs), VolSDF (smooth filled shapes), naive NeuS (dark *shells* on the outer boundary) and NeuS (density only on the first surface crossed). Slide 48 shows, for a ray,

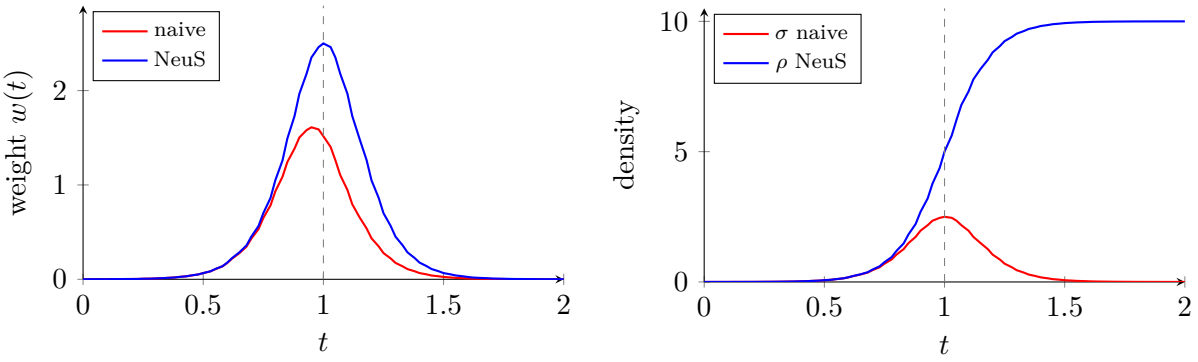


Figure 11: Original numerical illustration of slide 46 (ray crossing a plane at $t^* = 1$, $s = 10$, $f(t) = 1 - t$). Left: the naive weight peaks before the surface, the NeuS weight peaks exactly at it. Right: the densities.

the density $\sigma(t)$, the weight $w(t) = T\sigma$ and a zoom of the weight at the first surface for each method.

[PLACEHOLDER IMAGE]
Insert the image from **slide 47**
SDF of disc+rectangle (left); density maps of NeRF, VolSDF, naive NeuS, NeuS (right)

[PLACEHOLDER IMAGE]
Insert the image from **slide 48**
Plots of density, weight and zoomed weight along the ray for the four methods

20 Volume Rendering for Implicit Surfaces: results *(slides 50–54)*

All results are from NeuS (Wang et al., NeurIPS 2021): slide 50 compares a reference wooden box image with IDR, NeRF and "ours" geometry (NeuS is smooth and complete where IDR is wavy and NeRF noisy); slide 51 shows scissors and tools on a stone slab; slide 52 the half-sphere scene where IDR geometry is blobby; slide 53 a basket scene *without mask supervision*; slide 54 a gallery of reconstructions (Stonehenge blocks, fruit, owl, cans, plush toy, birds, cherub statue, skull).

[PLACEHOLDER IMAGE]
Insert the image from **slide 50**
Box: reference image, IDR, NeRF, NeuS

[PLACEHOLDER IMAGE]
Insert the image from **slide 51**
IDR / NeRF / NeuS geometry and NeuS rendering of scissors

[PLACEHOLDER IMAGE]
Insert the image from **slide 52**
IDR / NeRF / NeuS geometry and rendering of the half-sphere scene

[PLACEHOLDER IMAGE]
Insert the image from **slide 53**
Input subset, NeuS geometry and rendering of a basket, without mask supervision

[PLACEHOLDER IMAGE]
Insert the image from **slide 54**
Gallery of eight NeuS reconstructions (geometry + rendering)

Intuition

The comparison tells the whole story of the lecture: NeRF gives good images but bumpy geometry (density is unconstrained), IDR gives surfaces but struggles in optimisation (single-point rendering and masks), and NeuS combines volumetric rendering with an SDF-parametrised density to get both, even without masks.

21 Summary *(slides 55–56)*

Slide 55 recaps: *computing intersections of rays with implicit surfaces* with the analytic differentiation $\partial \boldsymbol{p}/\partial\theta = -\boldsymbol{\omega}(\nabla_{\boldsymbol{p}} f \cdot \boldsymbol{\omega})^{-1}\partial f/\partial\theta$, and *volume rendering with surface representations* via $\boldsymbol{x} \to f_\theta \to d \xrightarrow{\Psi} \sigma$. Slide 56 acknowledges the sources (adapted from CMU 16-825, Learning for 3D Vision, Prof. Shubham Tulsiani; secant slides by A. Geiger; sphere-tracing video by S. Lague; ray-marching figures by I. Gkioulekas; results from NGLOD, NeRF, Instant-NGP, NeRS, IDR, VolSDF, NeuS).

Comparison of the methods

	Implicit surface rendering (IDR)	**NeRF-like volume**	**VolSDF / NeuS**
Geometry	zero set of f_θ (SDF-like)	density σ + colour	SDF d mapped to density by Ψ / ρ
Points rendered per ray	one (the intersection)	many samples	many samples
Intersection / sampling	root finding, sphere tracing	stratified samples	stratified samples
Gradient	analytic via implicit differentiation; visibility discontinuities missed	standard backprop	standard backprop
Mask loss	critical	not needed	not needed (NeuS shown without masks)
Geometry quality	well-defined surface	does not necessarily capture geometry well	smooth, well-defined surface
Main drawback	hard optimisation, non-trivial differentiable rendering	noisy surface, overfitting risk	hand-designed conversion (heuristic; naive version biased)

Where the rendering weight goes

Key take-aways

- Volumes are expressive but under-constrained: good images do not imply good geometry. Restricting to *surfaces* (implicit zero sets, ideally SDFs with $\|\nabla f\| = 1$) injects the right prior.
- Ray–surface intersection is a 1-d root-finding problem; sphere tracing is safe for true SDFs, secant is a general fallback, and in practice they are combined.
- The intersection point is differentiable *analytically*: $\partial \boldsymbol{p}/\partial\theta = -\boldsymbol{\omega}(\nabla_{\boldsymbol{p}} f \cdot \boldsymbol{\omega})^{-1}\partial_\theta f$, independent of the solver and free of per-step memory, but it ignores visibility discontinuities and needs a mask loss.
- Volume rendering for implicit surfaces turns d into a density by a fixed map Ψ (VolSDF: Laplace CDF; NeuS: logistic, with a density that makes the weight $T\rho$ peak at the surface). Both are heuristics, and the naive logistic density is biased.

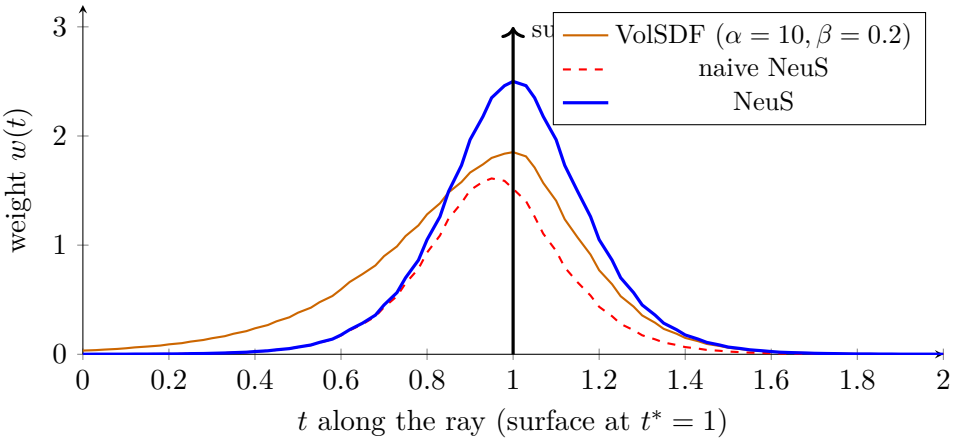


Figure 12: Original qualitative comparison for a ray crossing a single surface: surface rendering puts all weight on one point; volumetric methods spread it. NeuS is centred on the surface, naive NeuS is shifted in front of it, VolSDF depends on α, β and absorbs light slightly before the surface. (NeuS peak is $s/4 = 2.5$ for $s = 10$; curves are computed numerically.)

- Surface-based rendering allows physically based shading from normals (next lectures), while volumetric SDF methods are easier to optimise.

A Appendix: supplementary material (not on the slides)

The following derivations were added to make the notes self-contained; they do not correspond to specific slides.

A.1 From the continuous volume rendering integral to Eq. (1)

For a ray $\boldsymbol{x}(t)$, with density $\sigma(t)$ and emitted radiance $L_e(t)$, the continuous model is $L = \int_0^\infty T(t)\sigma(t)L_e(t)\,\mathrm{d}t$ with $T(t) = \exp(-\int_0^t \sigma(u)\,\mathrm{d}u)$. Assume $\sigma \equiv \sigma_i$ and $L_e \equiv L_{e,i}$ constant on segment $[t_i, t_i + \Delta t]$. Then $T(t) = T_i e^{-\sigma_i(t-t_i)}$ on it and

$$\int_{t_i}^{t_i+\Delta t} T(t)\sigma_i L_{e,i}\,\mathrm{d}t = T_i L_{e,i}(1 - e^{-\sigma_i \Delta t}),$$

which, summed over segments, is Eq. (1).

A.2 Ψ_β is a Laplace CDF

The Laplace density with scale β is $p(u) = \frac{1}{2\beta}e^{-|u|/\beta}$. For $s \le 0$, $\int_{-\infty}^{s} p = \frac{1}{2}e^{s/\beta}$; for $s > 0$, $\int_{-\infty}^{s} p = \frac{1}{2} + \int_0^s \frac{1}{2\beta}e^{-u/\beta}\mathrm{d}u = 1 - \frac{1}{2}e^{-s/\beta}$. These are exactly the two branches of Ψ_β, hence $\Psi_\beta(s) = \Pr[Z \le s]$, $\Psi_\beta(0) = \frac{1}{2}$, $\Psi'_\beta(s) = p(s)$, and by symmetry $\Psi_\beta(-d) = \Pr[Z \le -d] = \Pr[Z \ge d]$. As $\beta \to 0$, $\Psi_\beta(-d) \to \mathbf{1}[d < 0]$, i.e. $\sigma \to \alpha\,\mathbf{1}[\text{inside}]$.